

Proyecto 2

Juego en Assembly sobre Pochoco SoC

El proyecto considera una entrega en GitHub y una demostración en clases:

- **Fecha de entrega en GitHub:** 29 de septiembre.
- **Demostración en clases:** 30 de septiembre.

Utilizar los mismos grupos del laboratorio pasado.

Además de la entrega digital, cada grupo deberá realizar una demostración presencial durante la clase siguiente a la fecha de entrega.

1. Objetivo

Instanciar el **Pochoco SoC** en una FPGA **Lattice iCE40 HX1K** y desarrollar un juego interactivo en assembly para el **Espino Core**. El programa deberá traducirse a código máquina mediante un assembler desarrollado por el grupo y cargarse en la memoria del SoC.

2. Arquitectura base y estructura del proyecto

El punto de partida será el repositorio https://github.com/nic0villegasc/pochoco_soc. Usted es libre de realizar todas las modificaciones que quiera al RTL para optimizar el funcionamiento.

Cada grupo deberá crear un módulo superior propio, por ejemplo `game_top.v`, que instancie el SoC y seleccione la imagen de memoria del juego:

```
module game_top (... puertos de la Go Board ...);
  pochoco_soc #(.MemFile("sw/game.hex")) u_pochoco (
    .i_Clk(i_Clk),
    // conectar LEDs, botones, displays y SPI
  );
endmodule
```

El `game_top` es el *top-level* que se sintetiza y programa en la FPGA.

3. Juego de reflejos: requisito del proyecto

Todos los grupos deberán implementar un **juego de reflejos**. En cada ronda el sistema selecciona un LED objetivo de forma **pseudoaleatoria** y lo enciende. El jugador debe presionar el botón asociado a ese LED lo más rápido posible.

El juego debe seguir la secuencia siguiente:

- al iniciar cada ronda, encender los cuatro LEDs durante **al menos tres segundos**;
- seleccionar uno de los cuatro LEDs y encender únicamente el LED objetivo;
- iniciar la medición de tiempo en el mismo instante lógico en que comienza la ronda;
- esperar una pulsación; si el botón coincide con el LED objetivo, terminar la ronda;
- calcular y mostrar en los displays el tiempo de respuesta en **décimas de segundo**;
- repetir hasta completar diez respuestas correctas;
- después de la décima ronda, calcular y mostrar el tiempo promedio de respuesta en la misma unidad elegida;

- ante un botón incorrecto, indicar error y reiniciar la **ronda actual**; las rondas correctas acumuladas se conservan.

El tiempo se visualizará en **décimas de segundo**. La frecuencia de reloj y la conversión desde ciclos a décimas de segundo deben quedar explícitamente especificadas en el informe.

Para elegir el LED objetivo, se permite usar una semilla tomada del contador de ciclos y operaciones de assembly. La selección debe ser **perceptiblemente pseudoaleatoria**: no puede ser una secuencia fija que se repita exactamente en cada partida.

Para implementar la espera de tres segundos y medir la respuesta, el grupo deberá incorporar o reutilizar un contador de ciclos de reloj accesible por el programa. Toda la solución debe quedar documentada en el informe: arquitectura, modificaciones al RTL, temporización de tres segundos, generación pseudoaleatoria, unidad de tiempo, conversión desde ciclos y cálculo del promedio.

4. Assembler requerido

Cada grupo deberá implementar un assembler que reciba `game.s` y genere `game.hex`, con una palabra de 32 bits en hexadecimal por línea. El assembler puede desarrollarse en el **lenguaje que prefiera el grupo**, pero no puede invocar un assembler RISC-V externo.

El informe deberá documentar la sintaxis aceptada por el assembler, las instrucciones utilizadas por el juego, el tratamiento de etiquetas e inmediatos, y un ejemplo de la traducción desde assembly a código máquina.

5. Demostración en FPGA

Durante la demostración presencial se evaluará el funcionamiento en la FPGA y se realizarán preguntas sobre el diseño. El grupo deberá programar la Go Board y mostrar al menos:

- la fase inicial de tres segundos con los cuatro LEDs encendidos;
- una ronda correcta, verificando el LED objetivo y el tiempo mostrado;
- diez rondas correctas y el promedio resultante;
- una pulsación incorrecta y el reinicio de la ronda actual.

El README debe indicar el comando de síntesis/programación y cómo regenerar `game.hex` desde `game.s`.

6. Entrega

En la entrega se considerarán los siguientes ítems:

Ítem	Puntaje
Arquitectura del juego, contador de ciclos y temporización de tres segundos (PDF)	1,0
Assembly documentado, medición de tiempo y promedio de diez rondas (PDF)	1,0
Assembler: codificación, etiquetas, validación y pruebas (código + PDF)	1,0
Funcionamiento completo en la FPGA (presencial)	1,0
Preguntas orales del diseño (presencial)	2,0
Total	6,0

- Durante la evaluación se solicitará modificar una constante del juego, regenerar `game.hex` con el assembler y volver a demostrar el diseño. Además, los ayudantes preguntarán sobre el código del juego, assembler, cambios del procesador, etc.